

MVC Architecture Pattern

Structure of MVC
Motivation of MVC
MVC Example with Bouncing Ball

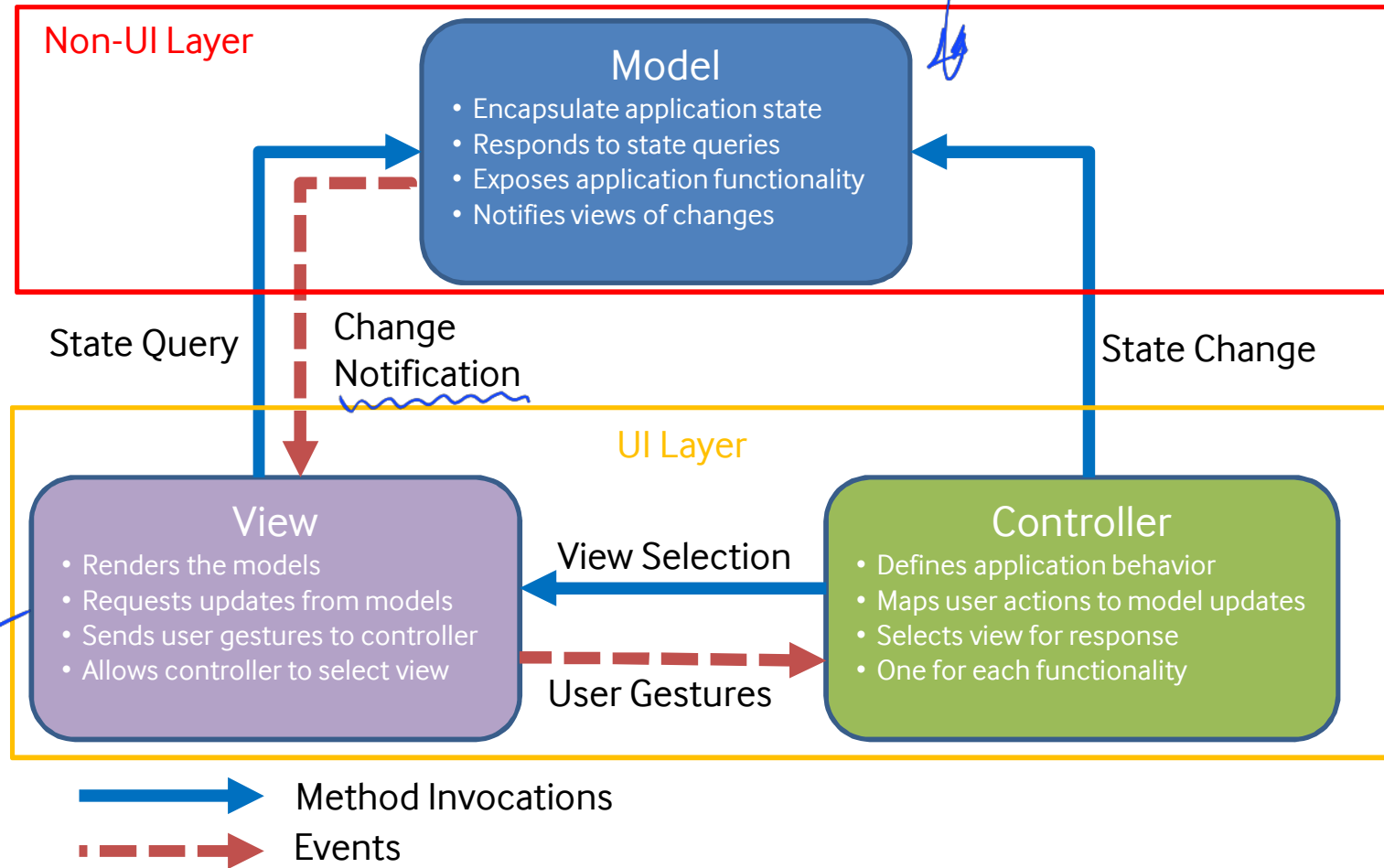
Introduction to Model-View-Controller Pattern

- Compound Pattern
 - A compound pattern combines two or more patterns into a solution that solves a recurring or general problem.
 - MVC uses different patterns
 - Strategy pattern
 - Observer pattern
 - Composite pattern
- MVC consists of three kinds of objects:
 - **Model** – the application object
 - **View** – UI (screen presentation)
 - **Controller** – defines the way the UI reacts to user inputs



MVC Structure

* Model은 view와 controller의 API를 호출하지 않는다.(반드시 지켜야함)



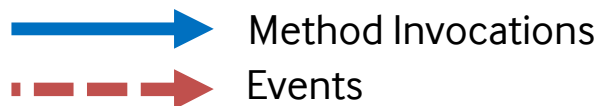
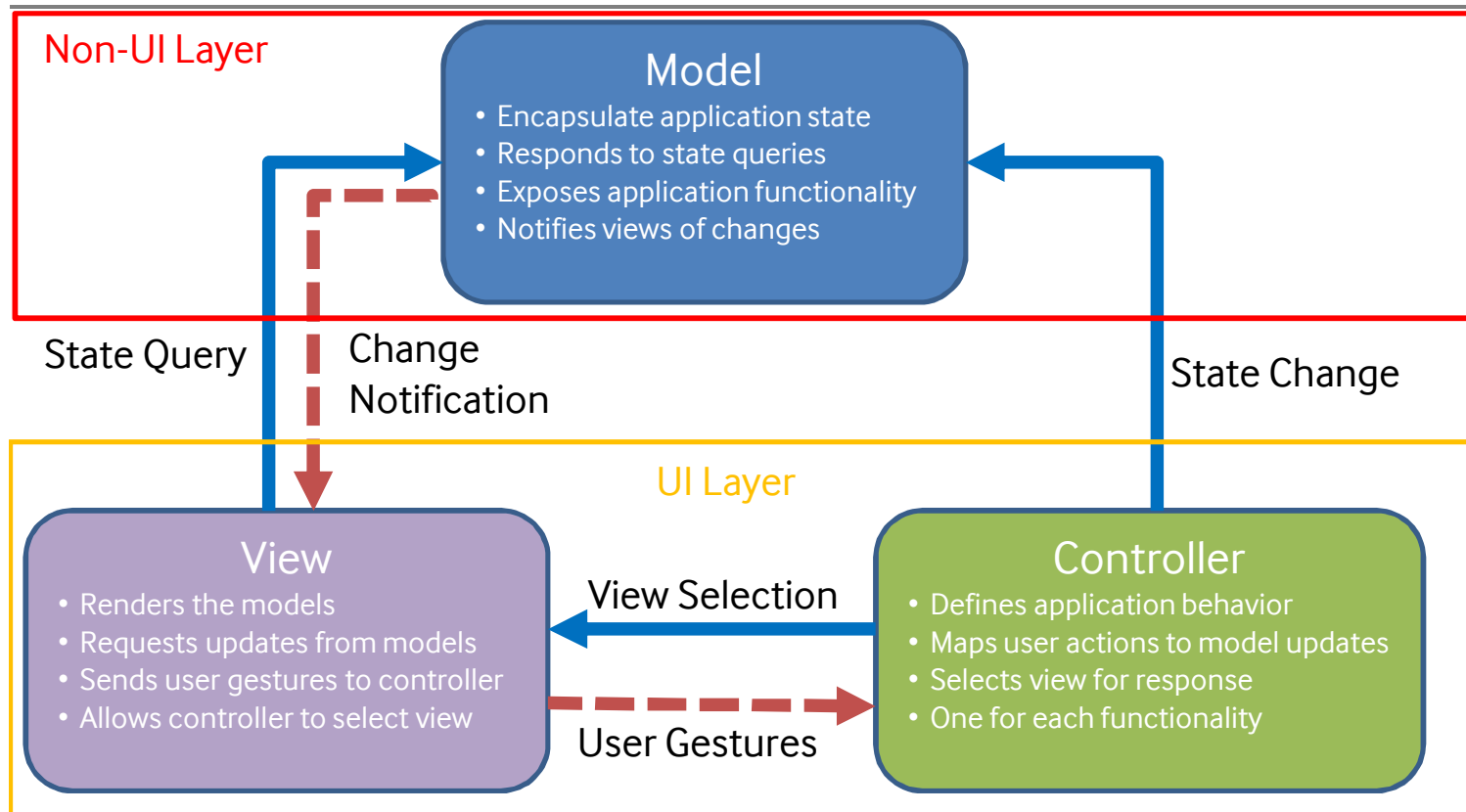
지켜받지 않음.

하에

능력.

지켜
받음.

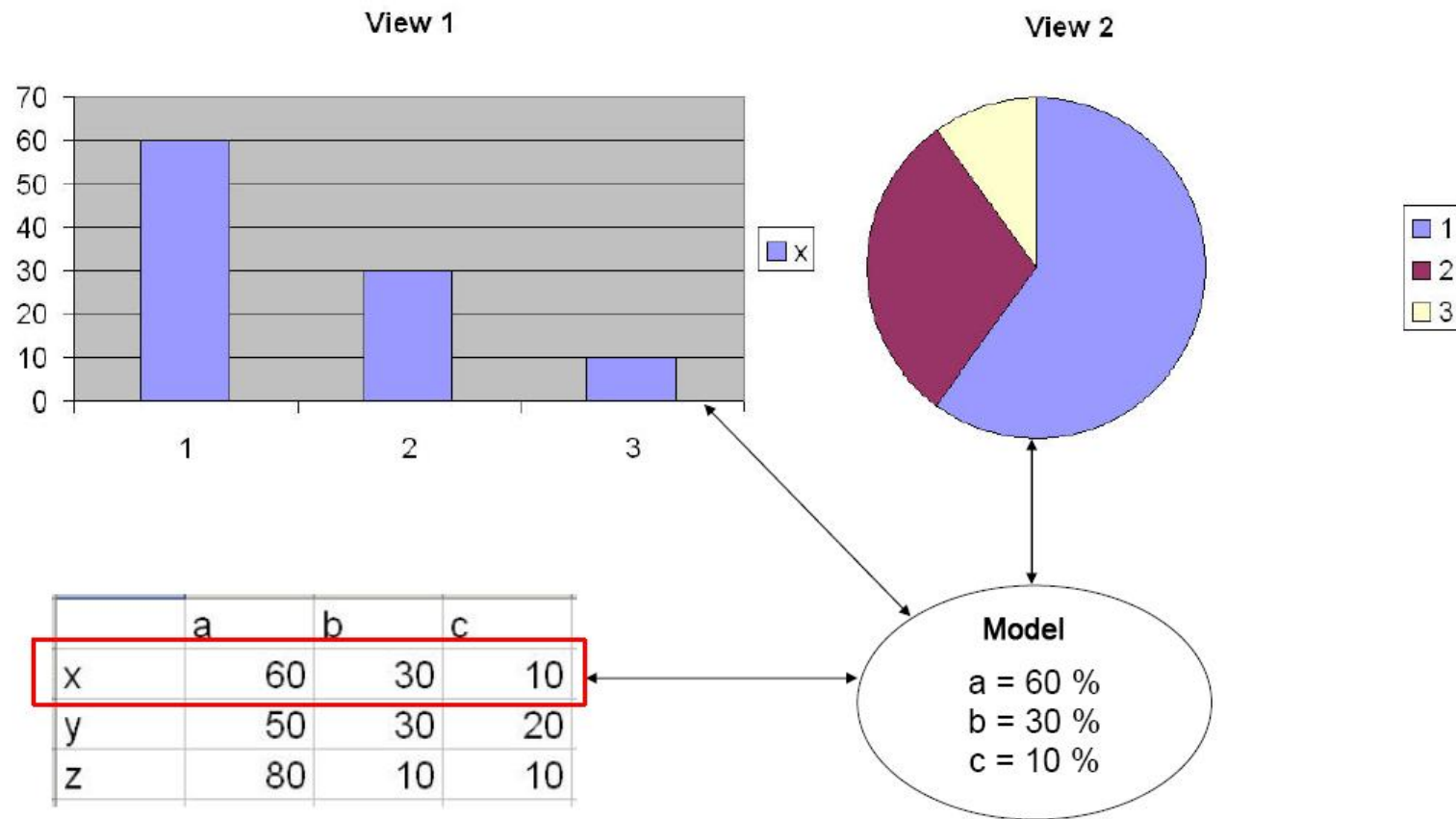
MVC Structure



Example Scenario

1. User interacts with the View
2. The controller handles the user input (often a callback function attached to UI elements)
3. The controller updates the model
4. The model notifies the view and it renders the changes
5. The view waits for user interaction

MVC Example



Motivations

- The same enterprise data needs to be accessed when **presented in different views**
 - e.g. HTML, JFC/swing, XML
- The same enterprise data needs to be **updated through different interactions**
- Supporting **multiple types of views and interactions should not impact** the components that provide the **core functionality** of the enterprise application

Solution

- **Separating responsibilities**

- **Separate core business model** functionality from the **presentation** and **control logic**
- Allows **multiple views** to share the same enterprise data model
- Makes supporting multiple clients **easier to implement, test, and maintain**

- **Model**

- Enterprise **data** and the business **rules**

- **View**

- **Renders** the contents of a model
- Accesses data through the **model**

- **Controller**

- **Translates** interactions with the **view** into actions to be performed by the **model**

Patterns in MVC

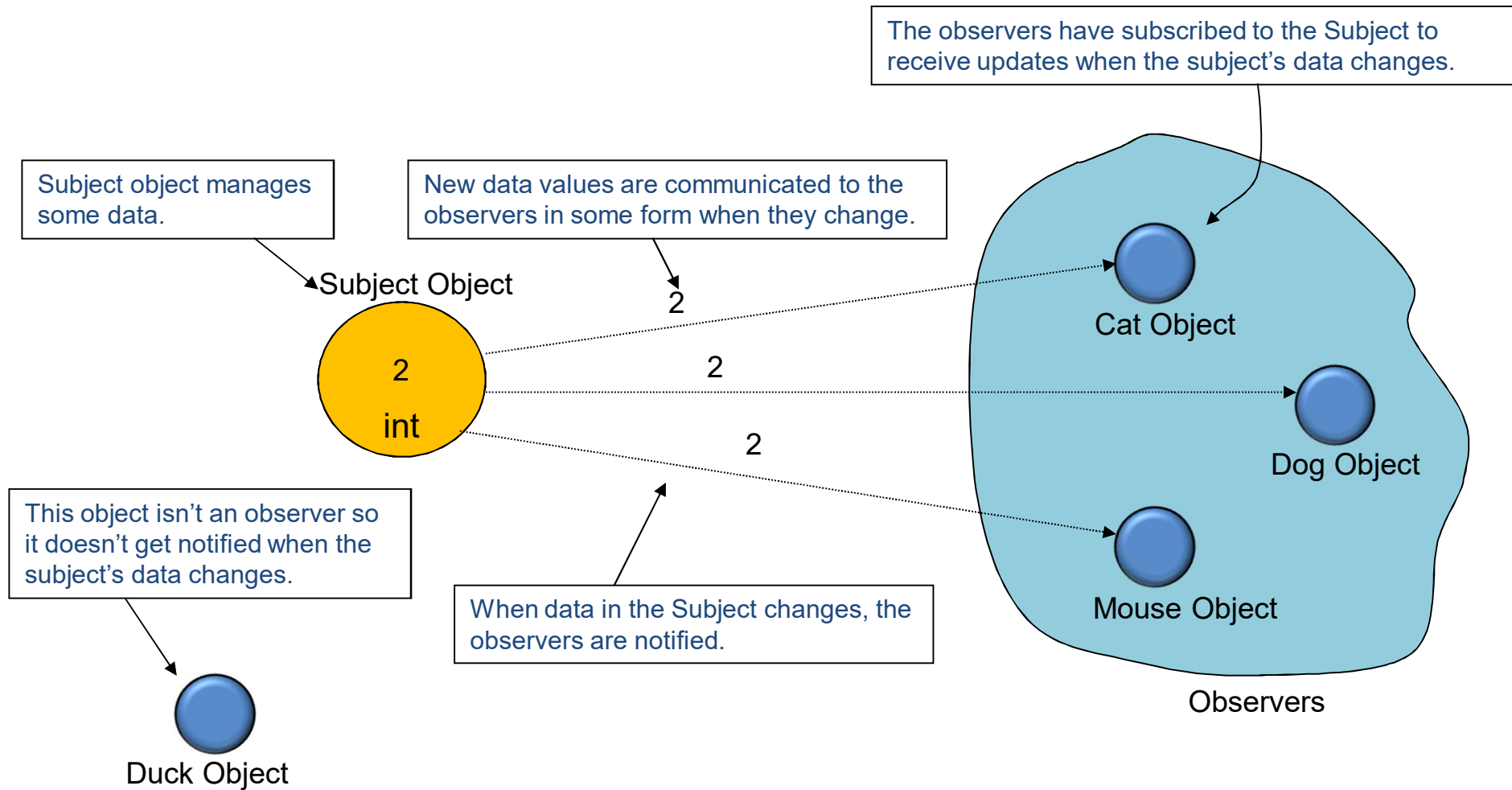
- Decouples views from models:
 - **Observer pattern**
 - Decoupling so that the **state change of the model** can be notified to **any number of objects** interested in the state **without** requiring the model to know **details of the objects**
- Nested view widgets:
 - **Composite pattern**
 - Grouping primitive and composite objects into new objects with identical interfaces: Composite View objects
- View delegates the gesture interpretation to controllers:
 - **Strategy pattern**
 - Controllers can be replaced to support different interpretations
- MVC uses **Factory** and **Decorator patterns** as well

Meet the Observer Pattern

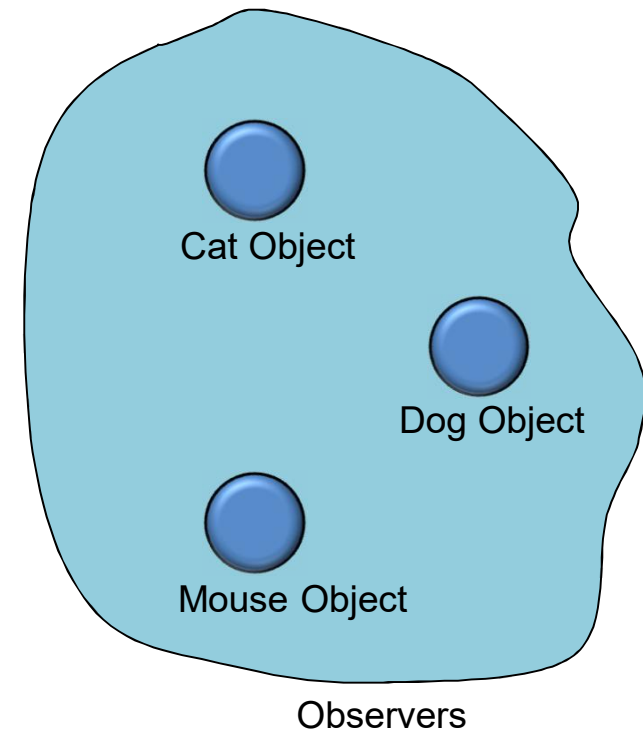
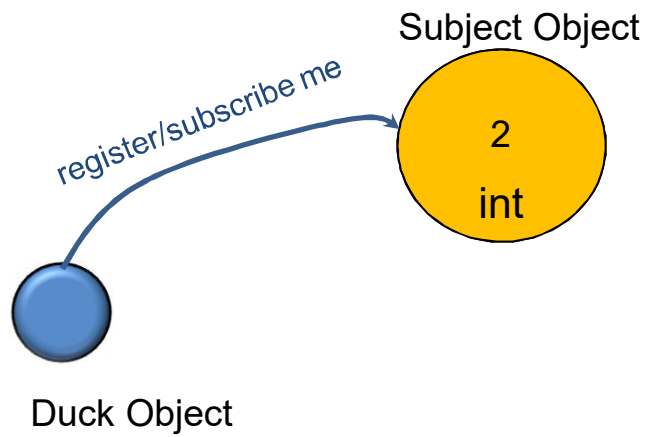
- Lets one or more objects be notified of state changes in other objects within the system
- Similar to newspaper publishing business
 - A news paper **publisher** goes into business and begins publishing newspapers
 - You **subscribe** to a particular publisher
 - Every time there's new edition it gets delivered to you. As long as you remain a subscriber; you get new newspapers
 - You **unsubscribe** when you don't want papers anymore
 - They stop delivery
 - While the publisher remains in business, people, hotels, airlines and other business **constantly subscribe and unsubscribe** to the newspaper.

Observer Pattern

- Publisher & Subscriber Model
 - Publisher : Subscriber = Subject : Observer

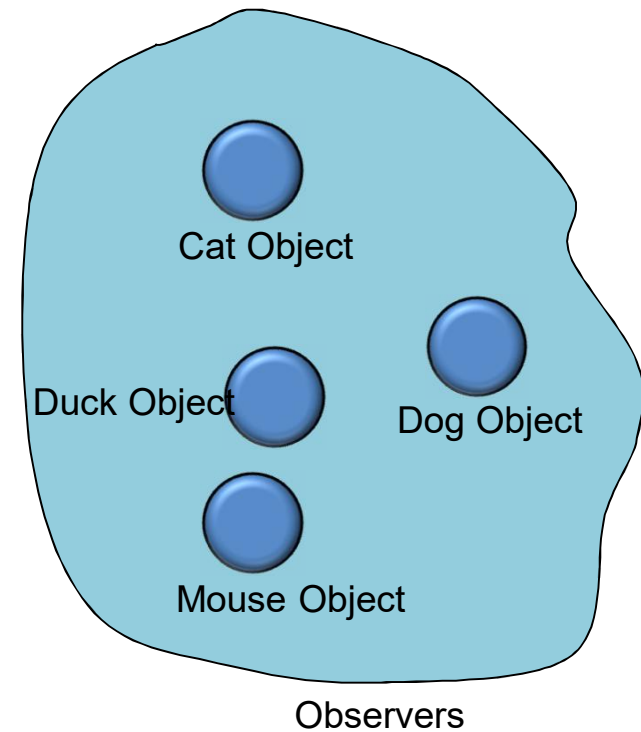
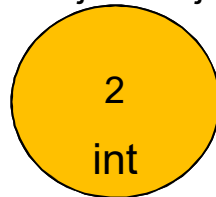


Duck wants to subscribe

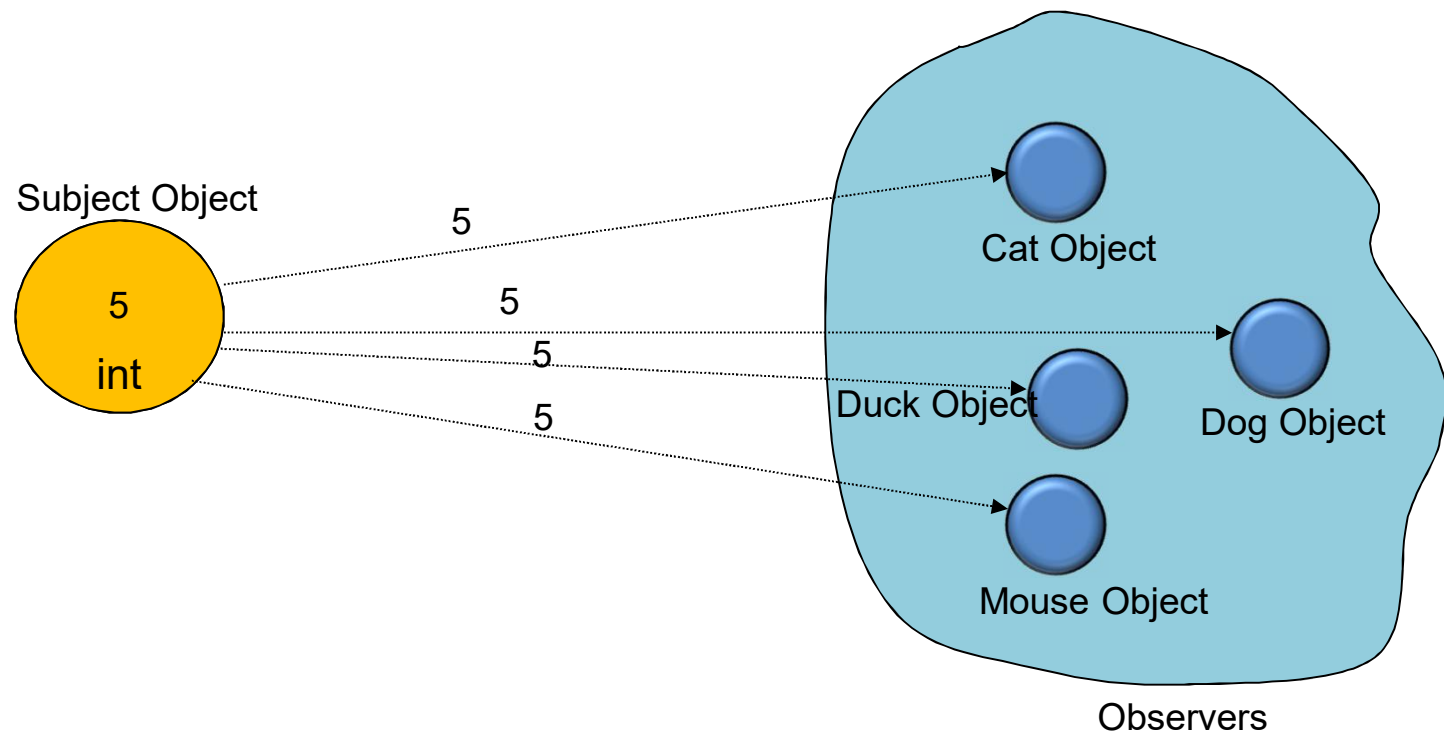


Duck is now an observer, too

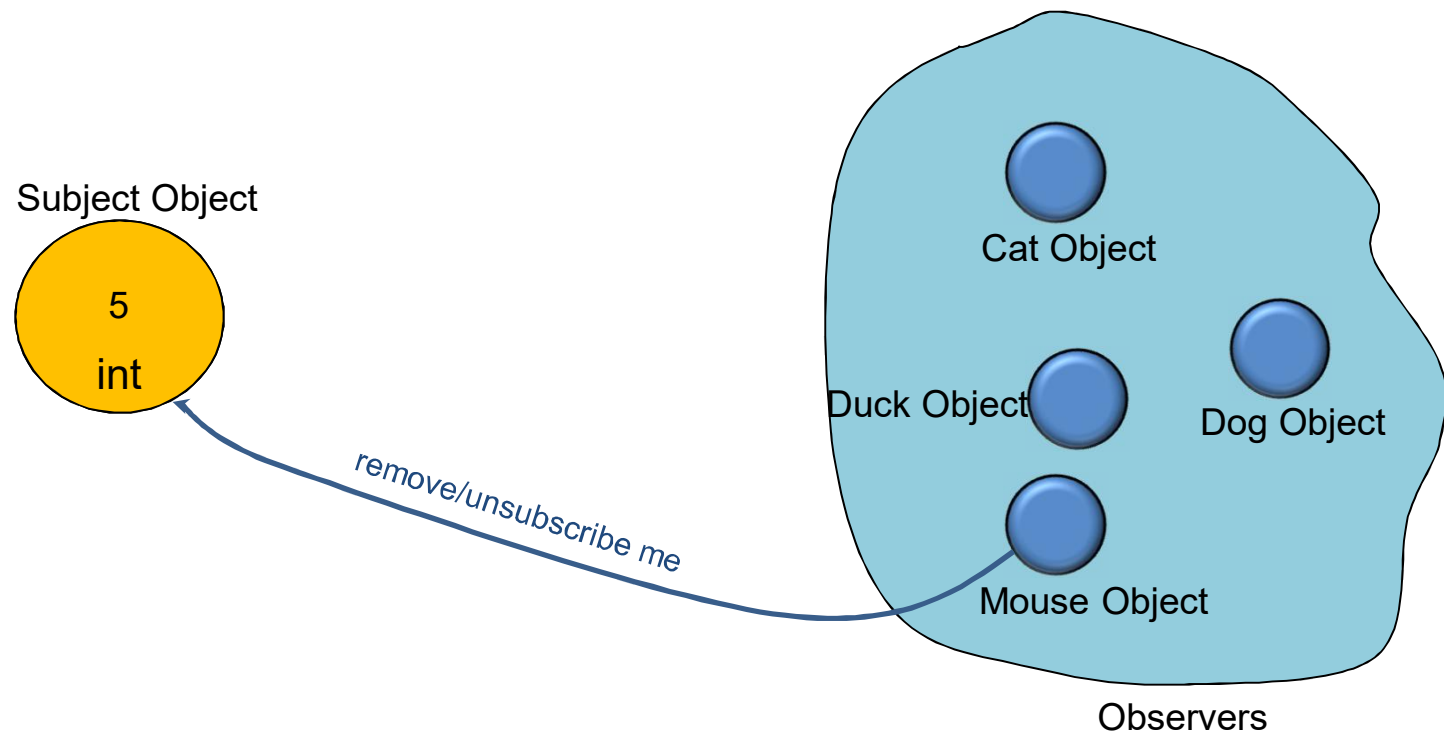
Subject Object



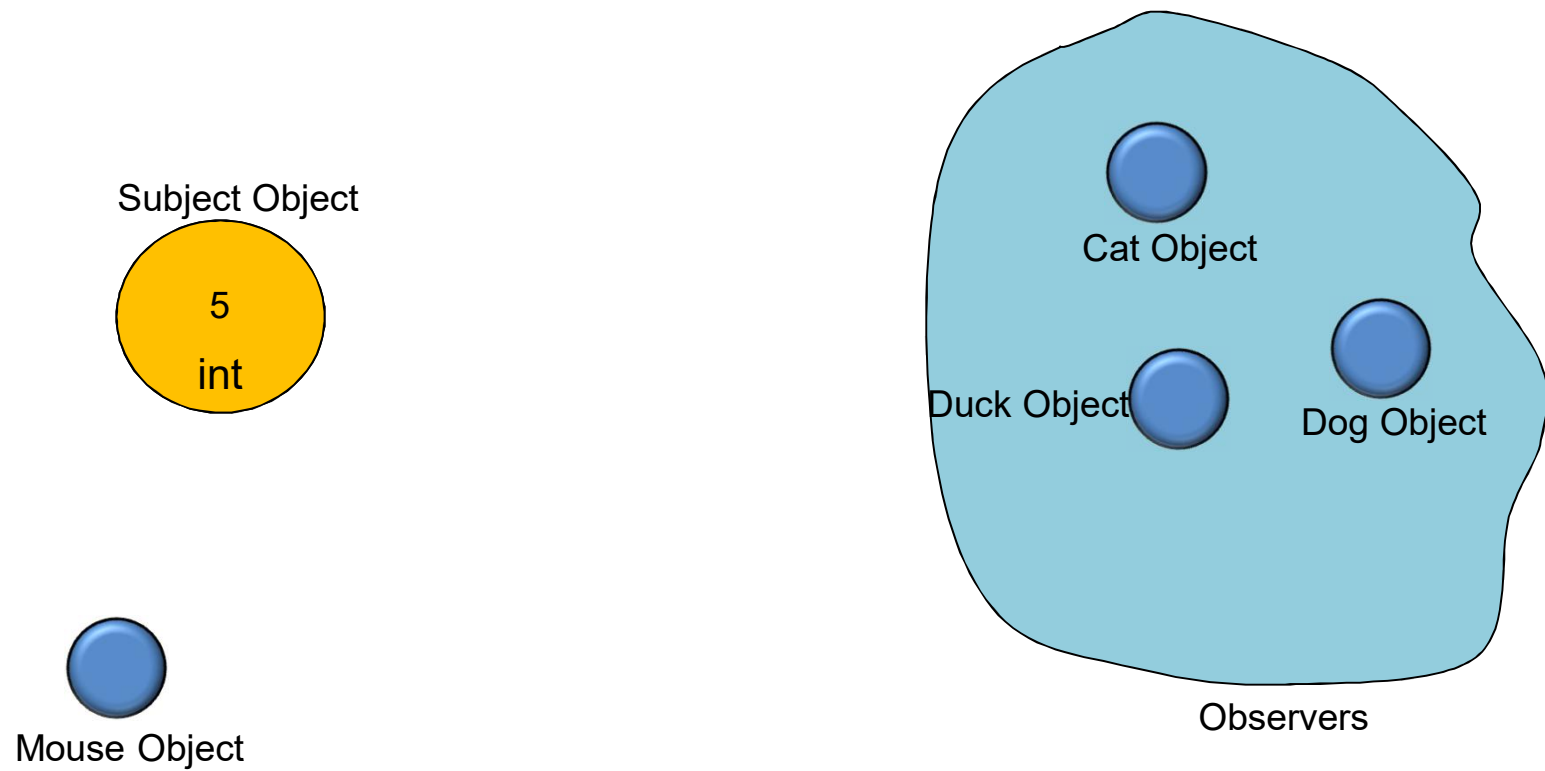
Notifying the observers



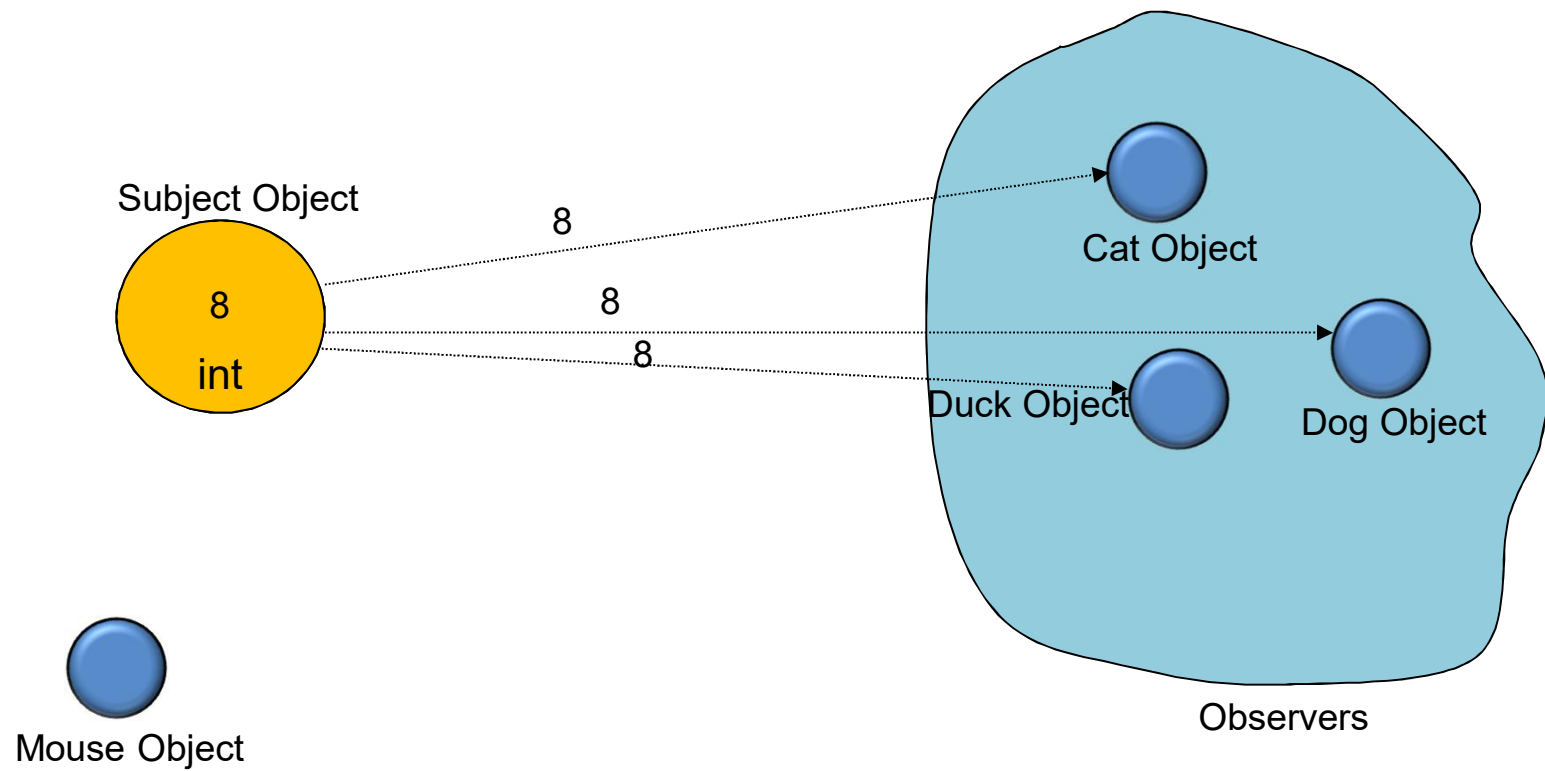
Mouse wants to unsubscribe



Mouse wants to unsubscribe

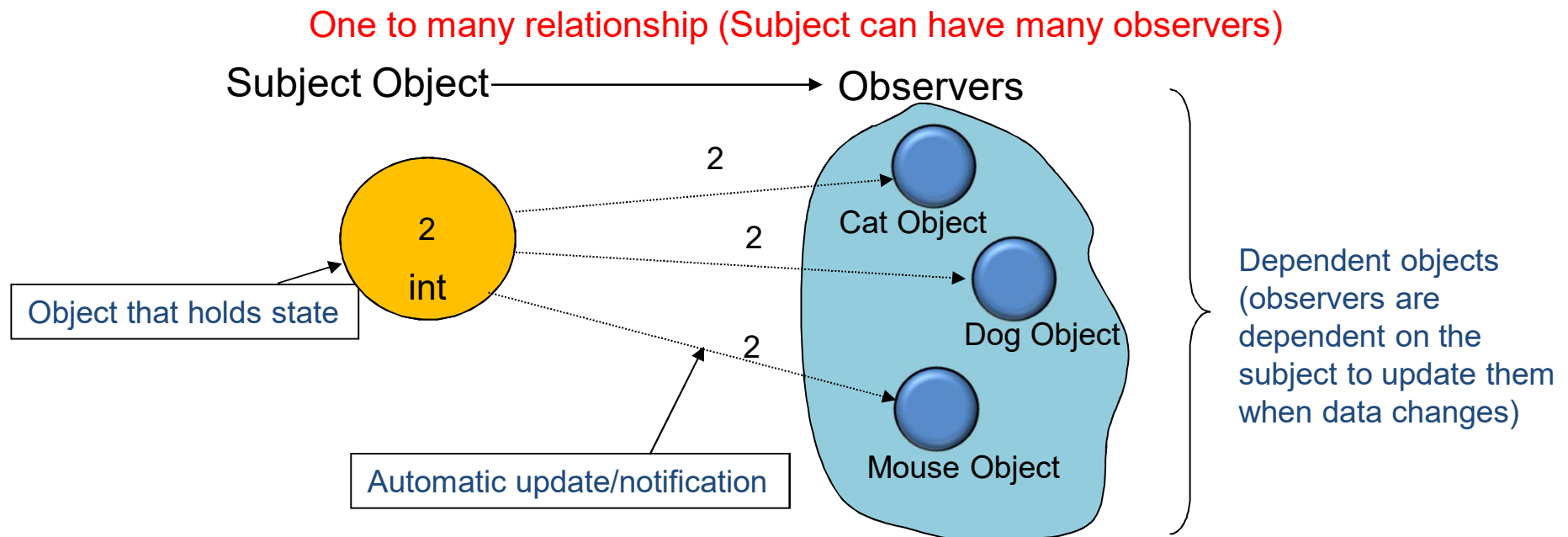


Notifying the observers

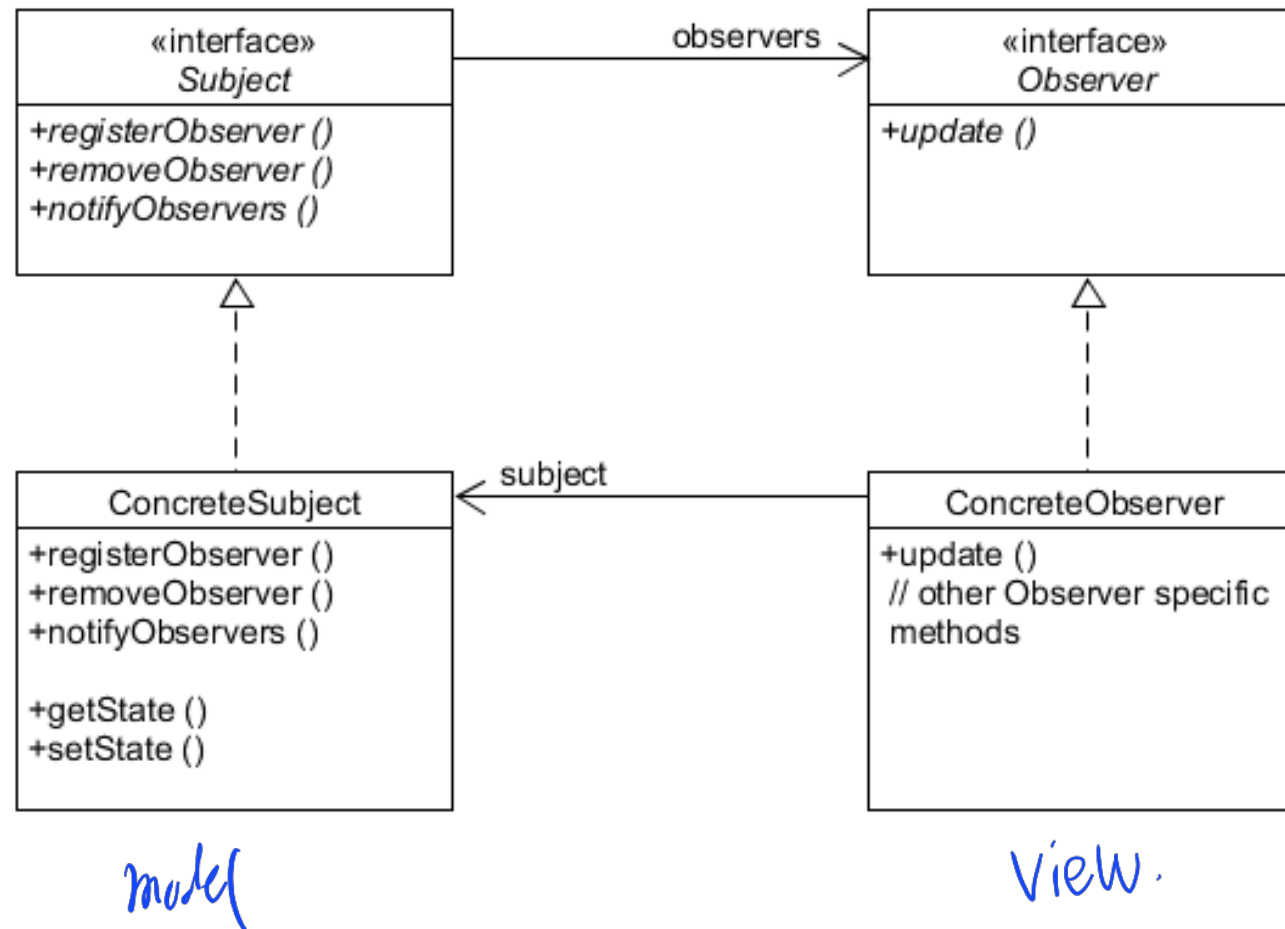


The Observer Pattern Defined

The **Observer Pattern** defines a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.



Class Diagram for Observer Pattern



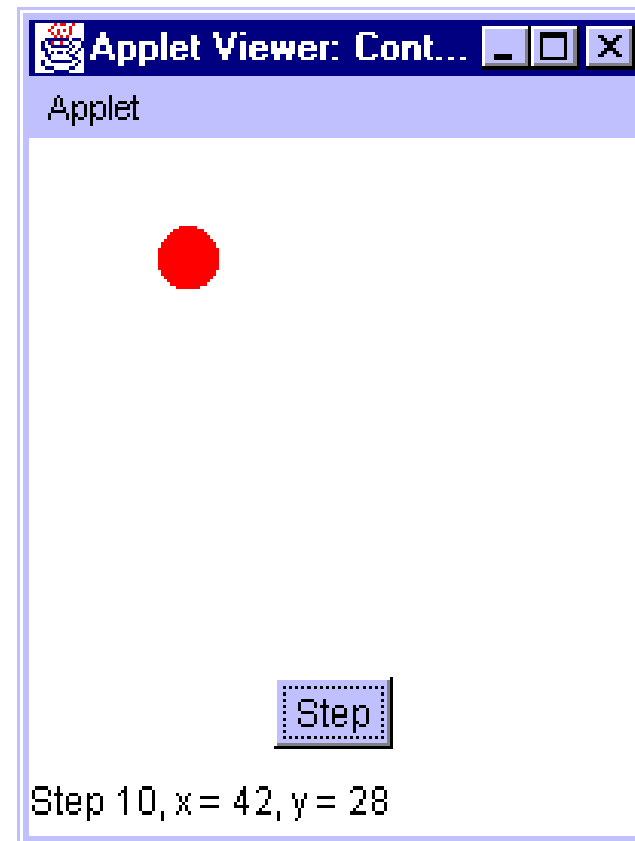
How to Use Observable Class in Java

- Observable (Publisher) side
 - To send notifications
 - Call `SetChanged()`
 - Call either `notifyObservers()` or `notifyObservers(Object arg)`
- Observer (Subscriber) side
 - To become an observer
 - Implement `java.util.Observer` interface
 - To subscribe
 - Call `addObserver()` on any `Observable` object
 - To unsubscribe
 - Call `deleteObserver()`
 - To get updated
 - Implement `update(Observable o, Object arg)`

✓ 5/13.

Example: Bouncing Ball Applet

- Each click of the Step button advances the ball a small amount
- The step number and ball position are displayed in the status line



The Ball Applet: Model

- The Ball Applet shows a ball bouncing in a window
- The Model controls the motion of the ball
- The Model must know the size of the window
 - so it knows when the ball should be made to bounce
- The Model does not need to know anything else about the GUI
- The Model is a Observable
 - `java.util` provides an **Observer** *interface* and an **Observable** *class*
 - An **Observable** is an object that can be “observed”
 - An **Observer** is “notified” when an object that it is observing announces a change

Model code

```
class Model extends Observable {
    final int BALL_SIZE = 20;
    int xPosition = 0;
    int yPosition = 0;
    int xLimit, yLimit;
    int xDelta = 6;
    int yDelta = 4;

    void makeOneStep() {
        xPosition += xDelta;
        if (xPosition < 0 || xPosition >= xLimit) {
            xDelta = -xDelta;
            xPosition += xDelta;
        }
        yPosition += yDelta;
        if (yPosition < 0 || yPosition >= yLimit) {
            yDelta = -yDelta;
            yPosition += yDelta;
        }
        setChanged();
        notifyObservers();
    } // end of makeOneStep method
} // end of Model class
```

The Ball Applet: View

- Needs access to the ball's state
 - in this case, its x-y location
- Should draw a ball
- The View is a Observer
- The View displays what is going on in the model
 - It should never display what *should* be going on in the model
 - For example, if you ask to save a file, the View shouldn't itself tell you that the file has been saved—it should tell you what the model reports

View code

```
class View extends Canvas implements Observer {
    Controller controller;
    Model model;
    int stepNumber = 0;

    View (Model model) {
        this.model = model;
    }
    public void paint(Graphics g) {
        g.setColor(Color.red);
        g.fillOval(model.xPosition, model.yPosition, model.BALL_SIZE,
            model.BALL_SIZE);
        controller.showStatus("Step " + (stepNumber++) +
            ", x = " + model.xPosition +
            ", y = " + model.yPosition);
    } // end paint method

    public void update(Observable obs, Object arg) {
        repaint();
    }
} // end class
```


The Ball Applet: Controller

- Controller tells the Model what to do
- Controller tells the View when it needs to refresh the display

Controller code

```
public class Controller extends JApplet {
    JPanel buttonPanel = new JPanel();
    JButton stepButton = new JButton("Step");
    Model model = new Model();
    View view = new View();

    public void init() {
        setLayout(new BorderLayout());
        buttonPanel.add(stepButton);
        this.add(BorderLayout.SOUTH, buttonPanel);
        this.add(BorderLayout.CENTER, view);
        stepButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent event) {
                model.makeOneStep(); }
        });
        model.addObserver(view);
        view.controller = this;
    }

    public void start() {
        model.xLimit = view.getSize().width - model.BALL_SIZE;
        model.yLimit = view.getSize().height - model.BALL_SIZE;
        repaint();
    }
}
```

Notes

- Litmus test: Swapping out user interface
 - Can the model be used, without modification, by a completely different UI?
 - e.g. Swing vs console text interface
- Model can be easily tested with JUnit
- Model actions should be quick
 - GUI is frozen while model executes
 - Alternative: multithreading

Summary

- MVC pattern is composed of the Observer, Strategy, and Composite patterns.
 - Model: makes use of the **Observer** pattern so that it can keep observers updated, yet stay decoupled from them.
 - Other patterns: **Strategy** and **Composite**
- Important Decision
 - Changes to UI should not trigger changes to Non-UI Modules
→ Non-UI modules should not depend on UI modules

